Américo Silva

Labanta Academia

TIC001_5 - Desenvolvimento de Aplicações Web Full-Stack

UF1.1 - Otimização e Estruturação de Documentos Web

Introdução ao Git

Branches Commits Repositórios Merge

O que é Git?

Sistema de Controlo de Versões Distribuído

Regista todas as modificações no código, permitindo reverter para versões anteriores.

Acompanhamento de Histórico

Mantém registo de quem fez o quê, quando e porquê, facilitando a auditoria.

Trabalho em Equipa

Facilita a colaboração entre desenvolvedores, permitindo trabalho em projeto sem sobreescrita.

Reverter Mudanças

Oferece capacidade de desfazer alterações específicas ou reverter para um estado anterior.

Porquê Usar Git?

Histórico Seguro

Garante que todas as versões do projeto são armazenadas de forma segura, protegendo contra a perda acidental de dados.

Facilita Colaboração

Permite que vários desenvolvedores trabalhem em paralelo no mesmo código, integrando as suas contribuições de forma eficiente.

Evita Perda de Ficheiros

Ao manter um histórico completo de todas as alterações, o Git atua como uma rede de segurança, permitindo recuperar versões anteriores.

Organização do Trabalho

Ajuda a manter o projeto organizado com um registo claro das alterações e a capacidade de gerir diferentes funcionalidades em ramos separados.

Git vs GitHub

VS

Git

- Software de **controlo de versões distribuído** instalado localmente no computador
- Regista todas as modificações no código num **histórico local**
- Permite criar **branches** para desenvolvimento isolado
- Operações de **commit, merge e rebase** executadas localmente

GitHub

- Plataforma online que **hospeda repositórios Git**
- Adiciona **funcionalidades sociais** como issues, pull requests e forks
- Fornece **gerenciamento de projetos** com projetos, milestones e actions
- Facilita a **colaboração** entre múltiplos desenvolvedores

Git é o software, GitHub é a plataforma. Existem outras plataformas semelhantes como GitLab e Bitbucket.

Instalação

Windows

1. Baixe o instalador do Git em **git-scm.com**
2. Execute o instalador e siga as instruções
3. Selecione as opções padrão para a maioria dos utilizadores

macOS

1. Baixe o instalador do Git em **git-scm.com**
2. Abra o ficheiro .dmg e arraste o Git para Applications
3. Adicione o Git ao PATH seguindo as instruções

Linux

1. Utilize o gestor de pacotes da sua distribuição:

Ubuntu/Debian: sudo apt install git

CentOS/RHEL: sudo yum install git

Arch Linux: sudo pacman -S git

Verificação

Após a instalação, verifique:

```
$ git --version
git version 2.39.2
```

Configuração Inicial

Após a instalação, é crucial configurar a sua identidade no Git. Esta informação será associada a todos os seus **commits** .

```
$ git config --global user.name "Seu Nome"
```

Define o seu nome de utilizador, que será exibido nos registos de commit.

O parâmetro `--global` aplica esta configuração a todos os repositórios.

```
$ git config --global user.email "email@example.com"
```

Configure o seu endereço de email, que também será associado aos seus commits.

Este email será usado em todos os commits que criar.

Verificação:

```
$ git config --global user.name
Seu Nome
$ git config --global user.email
email@example.com
```

Criar um Repositório

terminal.sh - 42 linhas

```
$ mkdir meu_projeto
$ cd meu_projeto
$ git init
Initialized empty Git repository in
/home/usuario/meu_projeto/.git/
$
```

O que acontece quando executamos git init?

- Cria um subdiretório oculto `.git` no projeto
- Este diretório contém todos os metadados e objetos para o controlo de versão
- Prepara o projeto para ser monitorizado pelo Git

Estrutura criada pelo git init

- `.git` — Diretório oculto
- branches
- config
- HEAD

Passos para criar um repositório

- Criar uma pasta para o seu projeto
- Navegar até essa pasta no terminal
- Executar o comando `git init`
- Verificar o estado inicial com `git status`

Fluxo de Trabalho Básico

1. Modificar Ficheiros

Crie ou altere ficheiros no seu diretório.

```
echo "Nova linha" > ficheiro.txt
```

2. Adicionar à Área de Staging

Prepare as alterações para o próximo commit.

```
git add .
```

3. Confirmar Alterações

Registe as alterações no histórico do repositório.

```
git commit -m "Mensagem do commit"
```

O ciclo fundamental no Git: modificar, adicionar à

Compreendendo Commits

Snapshots do Projeto

Os commits são como fotografias do seu projeto em pontos específicos do tempo.

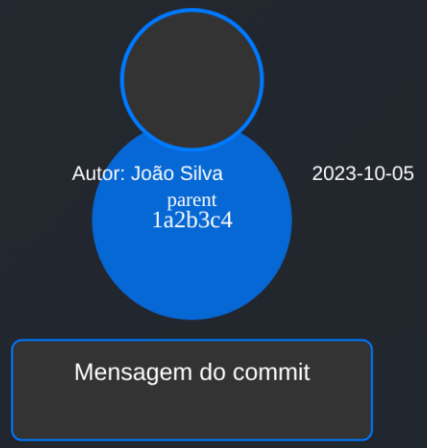
Mensagens Descritivas

Cada commit deve ter uma mensagem clara que explique o "porquê" da alteração.

Registo de Progresso

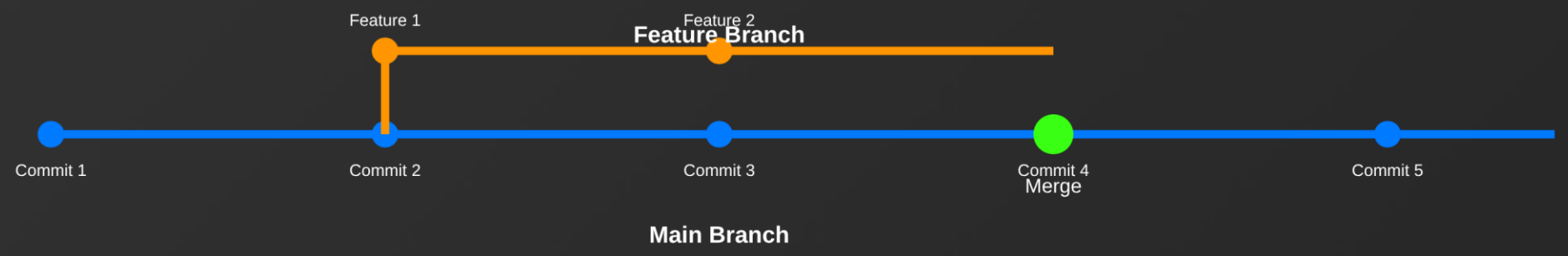
Os commits permitem registar o progresso incrementalmente, criando um histórico detalhado.

Visualização de um Commit




```
$ git commit -m "Adiciona nova funcionalidade"
[commit 1a2b3c4...] Adiciona nova funcionalidade
```

Trabalhando com Branches



Desenvolvimento Isolado

Permite trabalhar em novas funcionalidades sem afectar a linha principal.

Colaboração em Equipa

Facilita trabalho em paralelo permitindo que múltiplos desenvolvedores trabalhem simultaneamente.

Gestão de Versões

Permite manter diferentes versões simultaneamente, integrando-as quando prontas.

Segurança

Reduz riscos ao permitir testar alterações em isolamento antes de integrar.

Os branches são essenciais para um fluxo de trabalho eficiente no Git.

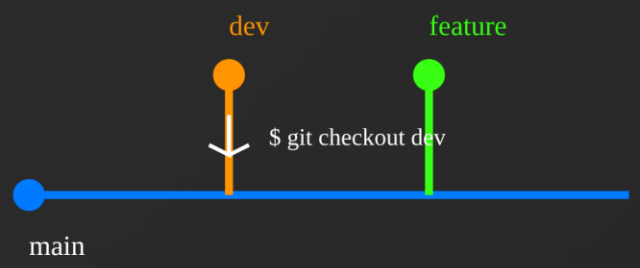
Criar e Mudar de Branches

```
git branch <nome-do-branch>
```
Cria um novo branch com o nome especificado

```
git checkout <nome-do-branch>
```
Muda para o branch especificado

```
git branch
```
Lista todos os branches existentes

Exemplo: git branch dev (cria um branch chamado "dev")



Dicas:

- Use `git branch -a` para ver todos os branches (locais e remotos)
- Use `git checkout -b <nome>` para criar e mudar para um novo branch de uma vez
- O branch principal geralmente chama-se `main` ou `master`

Fundir Branches

O que é um Merge?

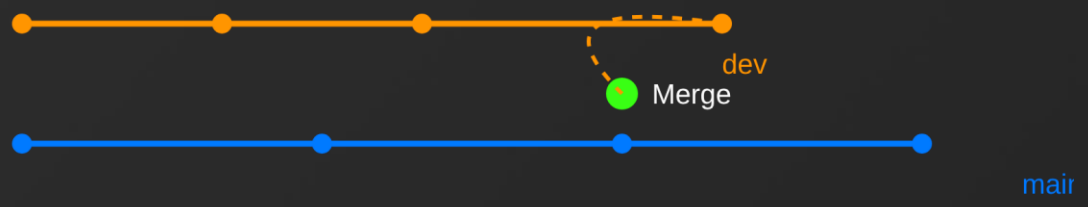
Integração de alterações de um branch noutro, combinando linhas de desenvolvimento.

Como Funciona

O Git cria um novo commit que une as históricas dos dois branches.

Conflitos

Ocorrem quando mesmas linhas são modificadas de forma diferente. Requer resolução manual.




```
$ git checkout main
```

```
$ git merge dev
```

```
$ git push origin main
```

Após merge, elimine o branch desenvolvido (git branch -d dev).

Outros Comandos Git Úteis

`git stash`

Guarda temporariamente alterações não confirmadas para mudar de contexto e retomá-las mais tarde.

`git rebase`

Reorganiza commits aplicando-os a outro branch, mantendo um histórico mais linear e limpo.

`git cherry-pick`

Aplica um commit específico de um branch para outro, permitindo selecionar alterações específicas.

`git tag`

Marca pontos específicos no histórico como importantes, normalmente usado para versões de lançamento.

`git fetch`

Baixa commits e referências do repositório remoto sem incorporar as alterações ao seu trabalho local.

`git reset`

Reverte o estado do branch para um commit anterior, útil para desfazer commits locais.

`git reflog`

Mostra um registro de todas as ações realizadas, permitindo recuperar commits "perdidos" após operações como reset.

`git blame`

Exibe quem modificou cada linha de um arquivo, quando e em qual commit, útil para rastrear alterações.

Estes comandos avançados aumentam significativamente a sua produtividade com Git

Exercício Prático

1 Criar Repositório

```
git init
```

2 Adicionar Ficheiros

```
git add .
```

3 Fazer Commits

```
git commit -m "Mensagem do commit"
```

4 Criar Branch

```
git branch dev
```

5 Mudar de Branch

```
git checkout dev
```

6 Fazer Merge

```
git checkout main && git merge dev
```

Simulação de Terminal

```
$ git init
Initialized empty Git repository in /path/to/repo/.git/
$ git checkout -b dev
Switched to a new branch 'dev'
$ git merge dev
Updating from 1234567 to 89abcdef
Fast-forward
```

Boas Práticas

Américo Silva

Labanta Academia

Mensagens de Commit

Escreva mensagens claras e concisas que expliquem o "porquê" da alteração.

```
$ git commit -m "feat: adicionar nova funcionalidade"
```

Nomenclatura de Branches

Use nomes descritivos e consistentes para branches, preferindo hífenes ou underscores.

```
$ git checkout -b feature/nova-funcionalidade
```

Organização de Workflow

Mantenha um histórico limpo e organizado, integrando alterações regularmente e evitando commits desnecessários.

Documentação

Adicione comentários úteis no código e mantenha documentação clara sobre o projeto e as decisões de design.

Conclusão

Controlo de Versões

Git regista todas as alterações no código ao longo do tempo, proporcionando um sistema robusto de controlo de versões.

Colaboração Eficaz

Facilita a colaboração em equipa, permitindo que múltiplos desenvolvedores trabalhem no mesmo projeto sem conflitos.

Fluxos de Trabalho

Os conceitos de branches e merges formam a base de fluxos de trabalho modernos de desenvolvimento.

"Git não é apenas uma ferramenta, é um paradigma de trabalho para desenvolvedores modernos."